

FILE PATH TRAVERSAL TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

Nivel: Practitioner

Tipo de explotación: Path Traversal mediante bypass de secuencias eliminadas de forma no recursiva

Materia: CNO IV WAP

Alumno: Gasca López Santiago Isaac – 184730

Docente: Mtro. Servando López Contreras



1. Marco Teórico

El control de acceso y el manejo seguro de archivos son pilares fundamentales en la arquitectura de aplicaciones web. Cuando una aplicación permite al usuario influir en la ruta o el nombre del recurso a cargar mediante parámetros de entrada (por ejemplo, a través de peticiones HTTP GET), surge el riesgo de Path Traversal (o Directory Traversal). Los desarrolladores suelen implementar filtros de saneamiento básicos que buscan y eliminan patrones maliciosos comunes, como la secuencia de escape de directorios `../`.

Sin embargo, si estos filtros operan mediante reemplazos simples de manera no recursiva (es decir, barriendo el texto una sola vez), el mecanismo de defensa puede ser evitado con facilidad mediante técnicas de ofuscación o estructuración anidada de payloads, comprometiendo la confidencialidad de los datos del servidor (violando el principio de la tríada CIA).

2. Objetivo

Demostrar y documentar la explotación exitosa de una vulnerabilidad de Path Traversal protegida por un filtro defectuoso no recursivo, utilizando herramientas de interceptación y manipulación de tráfico para extraer información confidencial del sistema operativo de la máquina víctima (PortSwigger Web Security Academy).

3. Vulnerabilidad

Nombre de la vulnerabilidad: File Path Traversal (Traversal sequences stripped non-recursively).

La aplicación web procesa parámetros de carga de imágenes de forma insegura y aplica un filtro que remueve las secuencias `../` de forma lineal. Al no ser un filtro recursivo, las modificaciones estructurales en el payload permiten burlar la validación y acceder a archivos restringidos del sistema central.

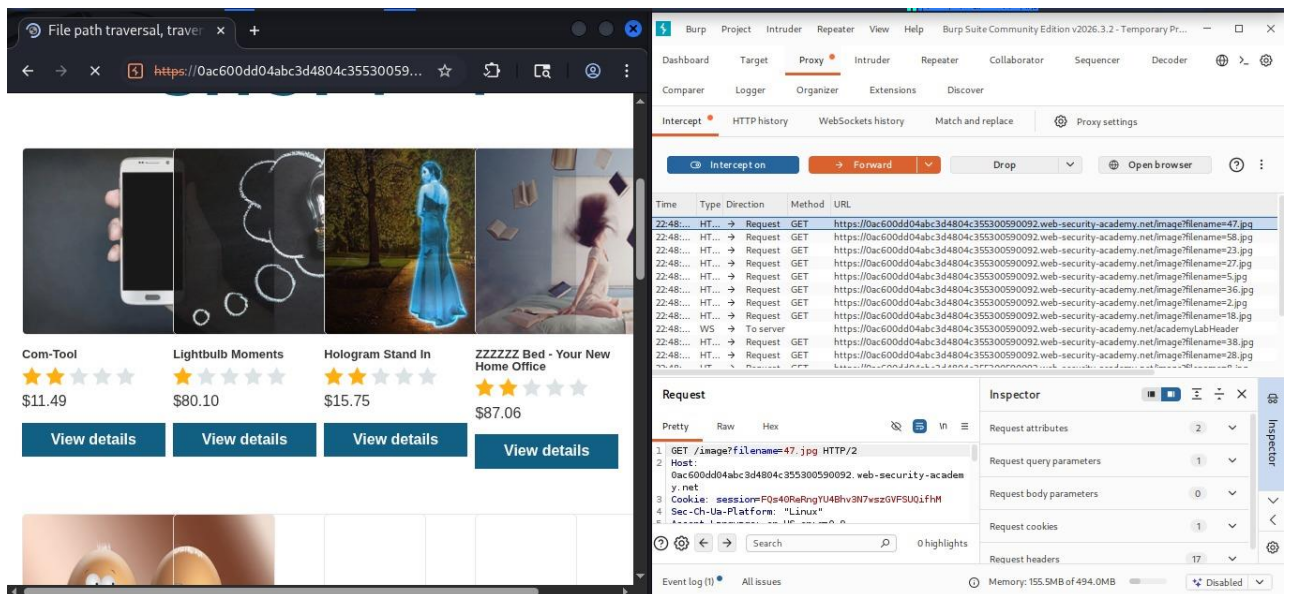
4. Procedimiento paso a paso

4.1 Configuración del entorno de interceptación:

Se configuró el proxy de Burp Suite asegurando que el interceptor estuviera activo, conectando el navegador al laboratorio de la academia de PortSwigger.

4.2 Generación de la petición base:

Se navegó por la plataforma web de la tienda y se seleccionó un producto para visualizar su imagen, generando una petición HTTP estándar del tipo GET /image?filename=X.jpg



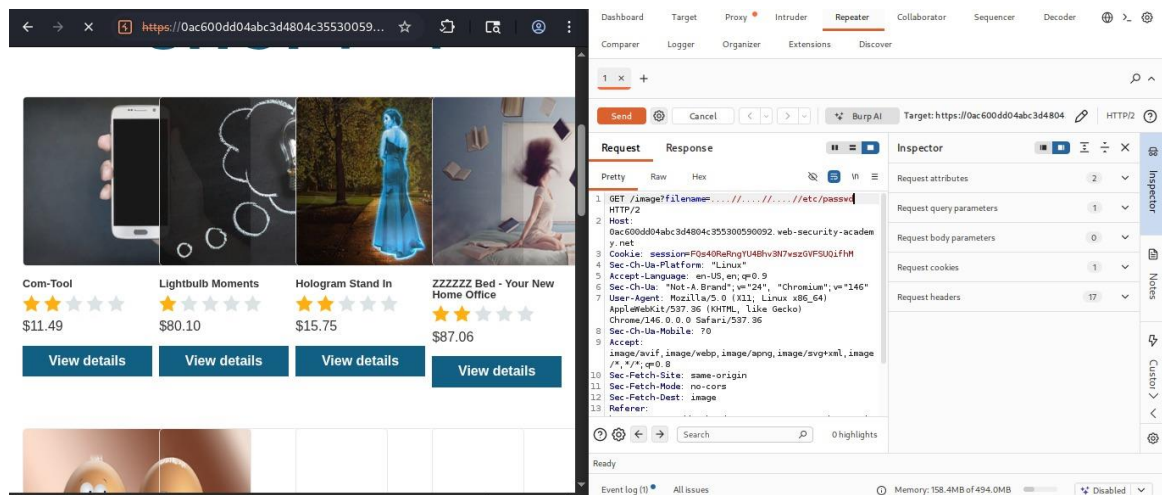
Panel de Burp Suite con la función Intercept is on activa y el historial HTTP mostrando la solicitud legítima de la imagen antes de su alteración.

4.3 Captura y envío a Repeater:

Desde el panel de intercepción y el historial de tráfico de Burp Suite (HTTP history), se localizó la petición y se envió al módulo Repeater para su análisis y manipulación controlada.

4.4 Inyección del payload de bypass:

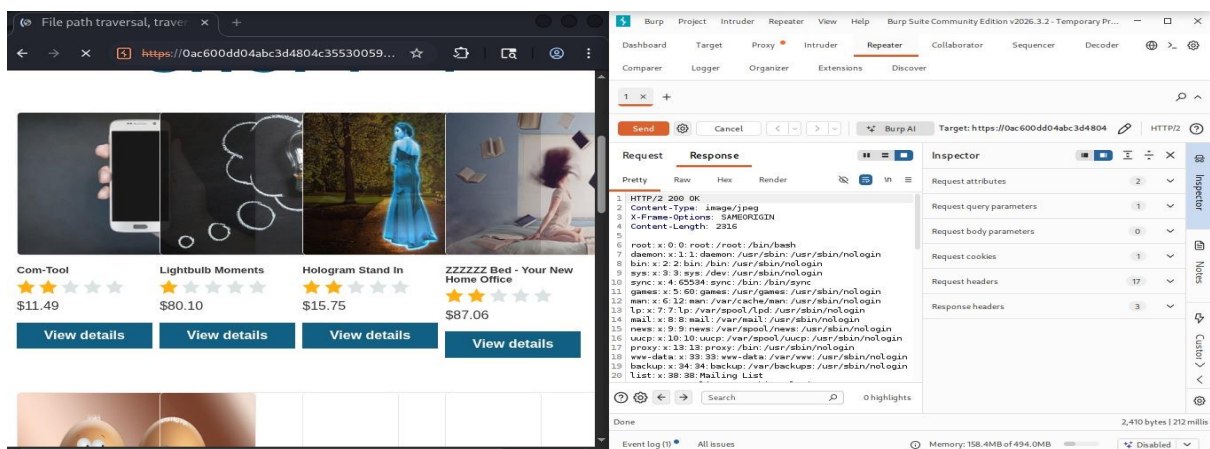
Se modificó el parámetro filename introduciendo una estructura anidada tolerada por el filtro no recursivo para forzar al sistema a resolver el archivo crítico del sistema operativo.



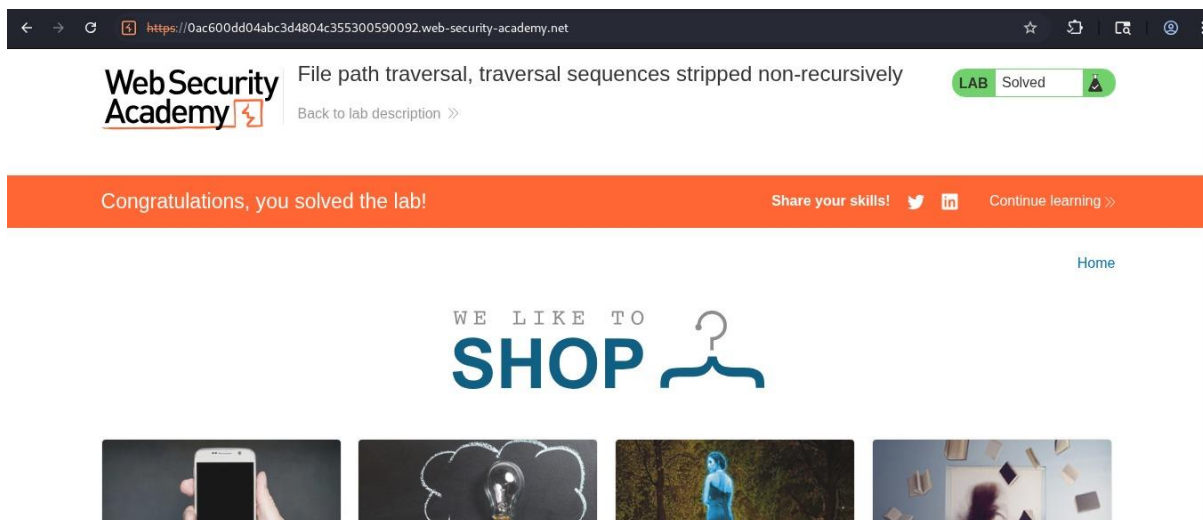
Pestaña Repeater de Burp Suite mostrando el parámetro del archivo modificado con el payload de bypass (.....//.....//etc/passwd) listo para el envío.

4.5 Validación del resultado y cierre del laboratorio:

Se reenvió la petición modificada analizando la respuesta del servidor y confirmando el éxito en la plataforma web.



Pestaña Response de Burp Suite reflejando el código HTTP 200 ok y la extracción correcta del archivo sensible del sistema Linux (root:x:0:0...).



Interfaz web de PortSwigger con la notificación en verde de "Lab Solved", validando la conclusión exitosa del ejercicio.

5. Payload utilizado

El payload inyectado en el parámetro de la petición HTTP fue:

- GET /image?filename=....//....//....//etc/passwd HTTP/2

6. Explicación Técnica del Payload

6.1 ¿Cómo opera el filtro del servidor?

El mecanismo de defensa busca la subcadena exacta `../` y, al encontrarla, la elimina de la petición una sola vez non-recursively.

6.2 ¿Por qué funciona el bypass?

Al enviar una estructura como `....//`, el filtro detecta y elimina los patrones interiores (o es incapaz de limpiar de forma recursiva los múltiples niveles). Como resultado, los caracteres restantes se reorganizan de manera automática al procesarse la cadena, reconstituyendo una secuencia válida de salto de directorio (`../`) que el sistema de archivos del sistema operativo interpreta y traduce fuera del directorio web raíz, permitiendo leer de forma íntegra el archivo `/etc/passwd`.

7. Mitigación

Para neutralizar de forma definitiva esta vulnerabilidad, se deben aplicar las siguientes directrices de desarrollo seguro:

7.1 Evitar el uso directo de entradas de usuario: No pasar nombres de archivo crudos proporcionados por el cliente directamente a las APIs del sistema de archivos.

7.2 Implementar validación por listas blancas (Allowlists): Permitir únicamente identificadores o nombres estrictamente predefinidos y acotados.

7.3 Uso de rutas canonicalizadas: En lenguajes de programación del servidor, validar mediante la función de ruta limpia (como `getCanonicalPath()` en Java), asegurando que cualquier intento de salir del directorio base sea detectado y bloqueado independientemente de los caracteres de escape utilizados.

8. Conclusión

La práctica permitió evidenciar que los mecanismos de saneamiento superficiales y no recursivos son completamente ineficaces frente a ataques avanzados de Path Traversal. La comprensión de la lógica de análisis de entradas y el uso de funciones de canonicalización robustas son indispensables para evitar la exposición de información crítica en entornos corporativos.

9. Fuente

PortSwigger Web Security Academy. *File path traversal, traversal sequences stripped non-recursively*. Recuperado de [PortSwigger Labs](#).

